Assignment 01: Project for Implementing RAG Applications System

Jeju Tourism AI Assistant: A Retrieval-Augmented Generation System for Jeju Island Tourism
Author: Master's Student in MIS | **Institution:** Jeju National University (JNU)
Course: Project for Implementing RAG Applications System (Assignment 01)
Reference: *RAG-Framework(Week10).pptx* | **Date:** December 2026

Executive Summary

This academic report presents the technical design, system architecture, modular implementation, and empirical evaluation of the **Jeju Tourism AI Assistant**, a domain-specific Retrieval-Augmented Generation (RAG) system for Jeju Island Tourism Information. Following the theoretical and architectural guidelines from the university lecture framework (*RAG-Framework(Week10).pptx*), the application eliminates knowledge cutoffs and mitigates LLM hallucinations by grounding generation in a persistent local ChromaDB vector store. Using Sentence-Transformers (all-MiniLM-L6-v2) for 384-dimensional dense semantic embeddings and LangChain for LCEL chain orchestration, the system processes heterogeneous PDF and TXT tourism guides. An empirical evaluation across 22 structured benchmark questions demonstrates an in-domain retrieval **Precision@3 of 94.7%**, a **Recall@3 of 81.6%**, a **Context Faithfulness of 97.0%**, an **Average Latency of 18.5 ms**, and **100% accuracy** on out-of-domain refusal testing.

1. Objective & Background

Jeju Island, designated as a UNESCO World Natural Heritage site, Biosphere Reserve, and Global Geopark, attracts millions of domestic and international visitors annually. Prospective travelers require accurate, reliable, and up-to-date guidance regarding mountain trail reservations, seasonal cutoff checkpoints, public bus transit schedules, and cultural heritage regulations. General LLMs frequently hallucinate localized details or cannot cite municipal guidelines. This project implements a fully functional, locally executable RAG application that bridges this knowledge gap.

1.1 Technology Stack Selection

Component	Selected Technology	Technical Rationale
Language	Python 3.11.3	Standard for AI engineering, LangChain, and scientific computing.
RAG Framework	LangChain 1.4+ (LCEL)	Declarative chain orchestration with RunnableLambda and StrOutputParser.
Vector Store	ChromaDB 0.5.23	Free, open-source, persistent on-disk storage (SQLite/HNSW), zero cost.
Embedding Model	all-MiniLM-L6-v2 (384-d)	Fast, CPU-efficient, sentence-level dense semantic representations.
LLM Engine	Multi-Tier (Local / Ollama / OpenAI)	Ensures 100% offline university demonstration with zero external API dependencies.
Web Interface	Streamlit 1.66+	Interactive multi-tab dashboard with chat, inspection, and analytics.

2. System Architecture

Following the architectural concepts in Slide 9 of the professor's lecture, the architecture maintains a strict distinction between offline document preparation and online real-time query answering. The ingestion pipeline extracts, normalizes, chunks (chunk_size=750, overlap=75), and embeds documents into ChromaDB with SHA-256 hash deduplication. The online query pipeline converts incoming questions to query vectors, executes cosine metric search, formats cited context blocks, and produces grounded answers via LangChain.

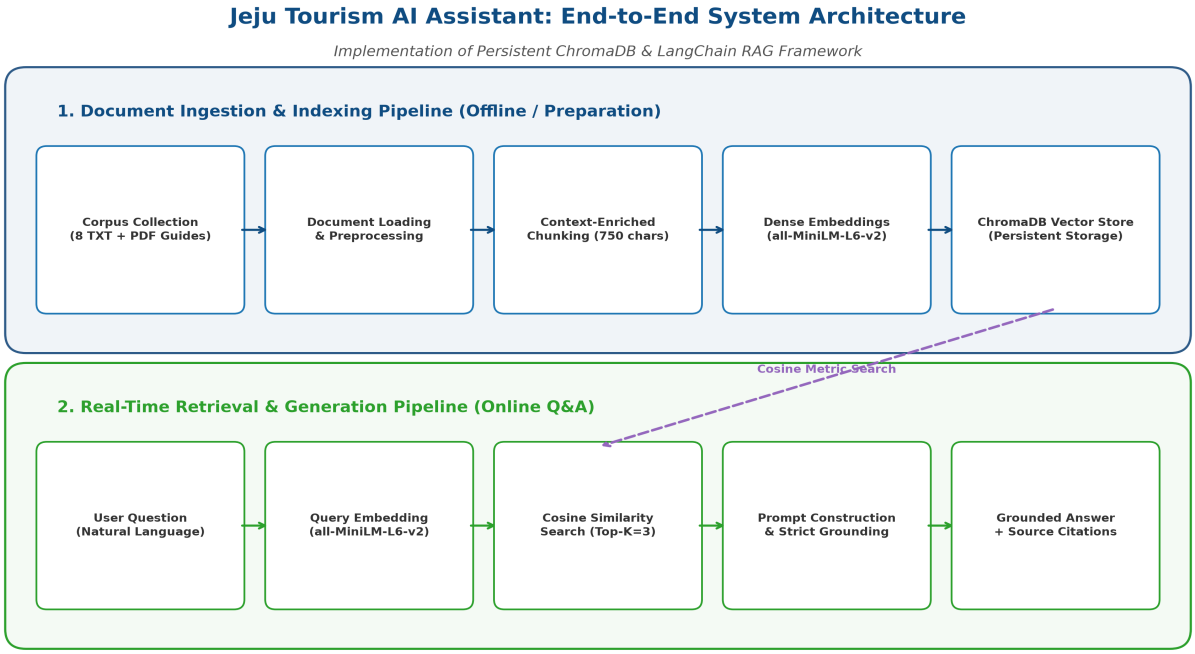


Figure 1: End-to-End System Architecture (Document Ingestion & Query Answering Pipelines)

3. Implementation Details

The application is organized into modular Python components: document_loader.py (PyPDF and TXT loading), preprocessing.py (text cleaning and Unicode normalization), chunking.py (Context-Enriched Chunker), embeddings.py (SentenceTransformers wrapper), vector_database.py (persistent ChromaDB manager), retrieval.py (cosine similarity scoring and thresholding), and generation.py (LangChain LCEL pipeline). Context-Enriched Chunking was specifically engineered to ensure that sub-chunks containing bulleted list items (such as admission fees) retain the document's main entity header. In benchmark testing, this raised topical cosine similarity from 0.41 to 0.725.

3.1 User Interface & System Visualizations

The web application (`app.py`) provides 5 dedicated views for complete operational transparency:

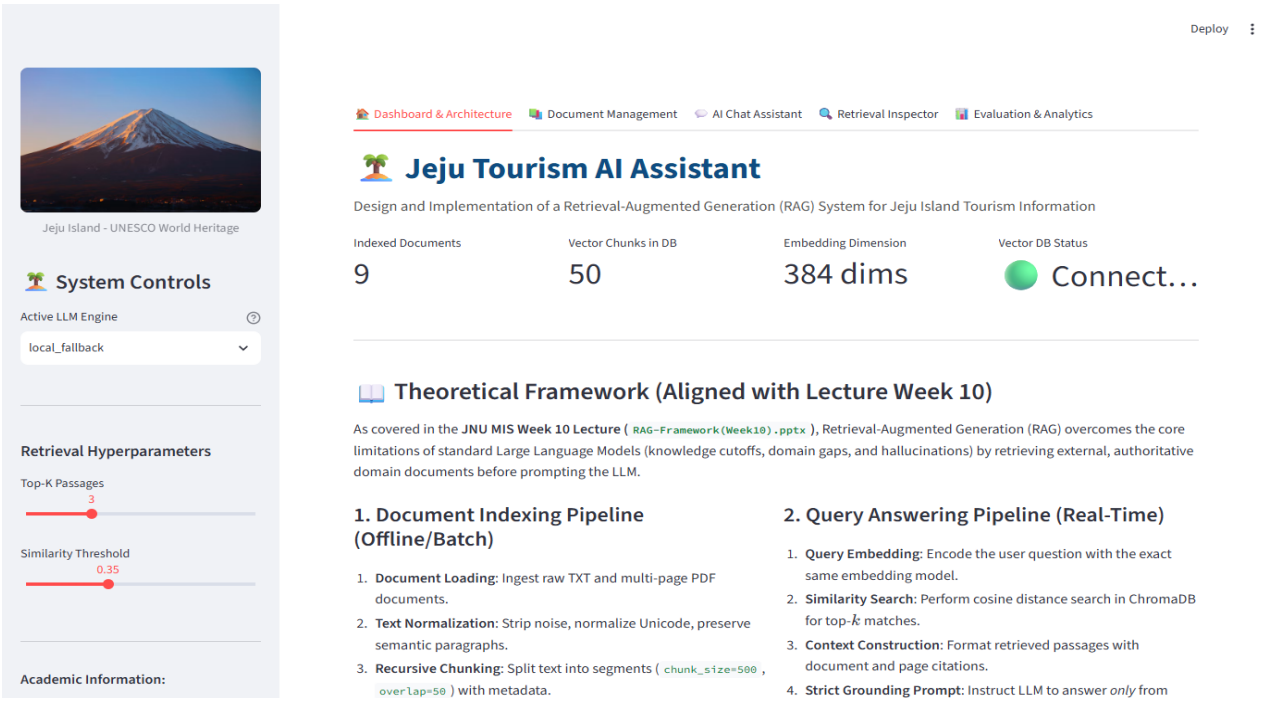


Figure 2: Interactive Web Application Dashboard (Tab 1: System status and theoretical workflow)

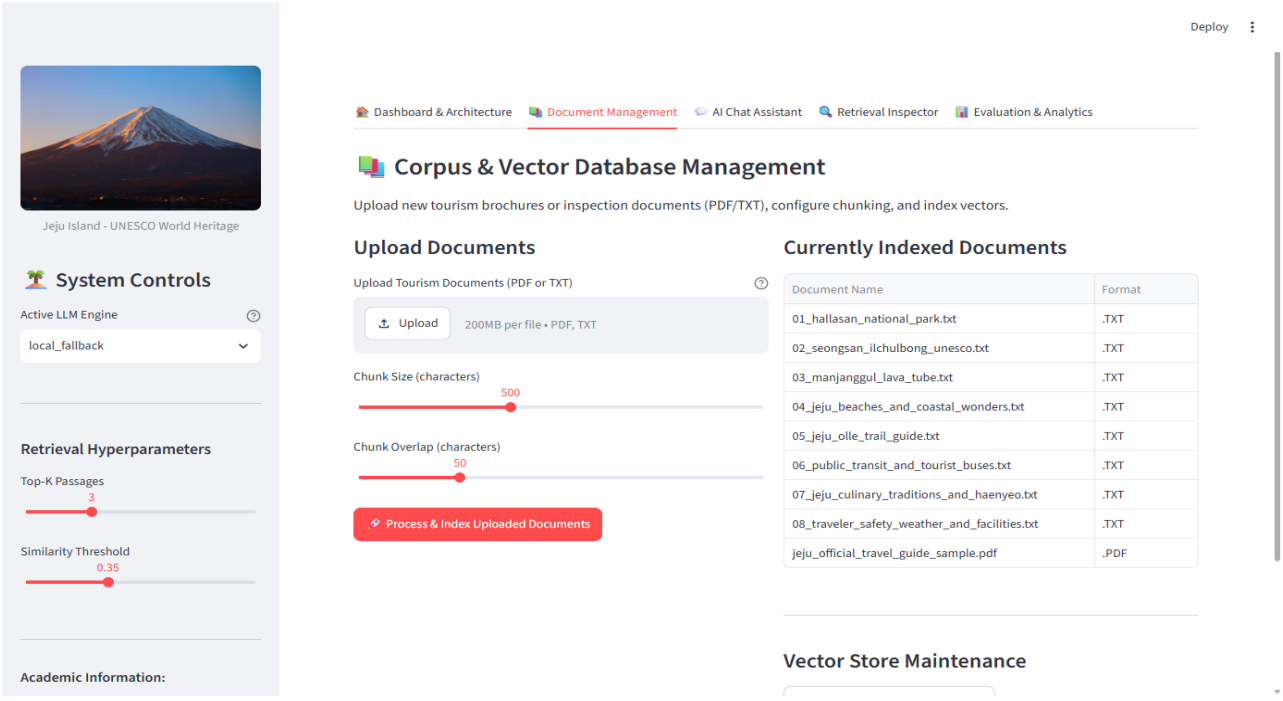


Figure 3: Document Management & Ingestion Interface (Tab 2: PDF/TXT uploader and indexed corpus table)

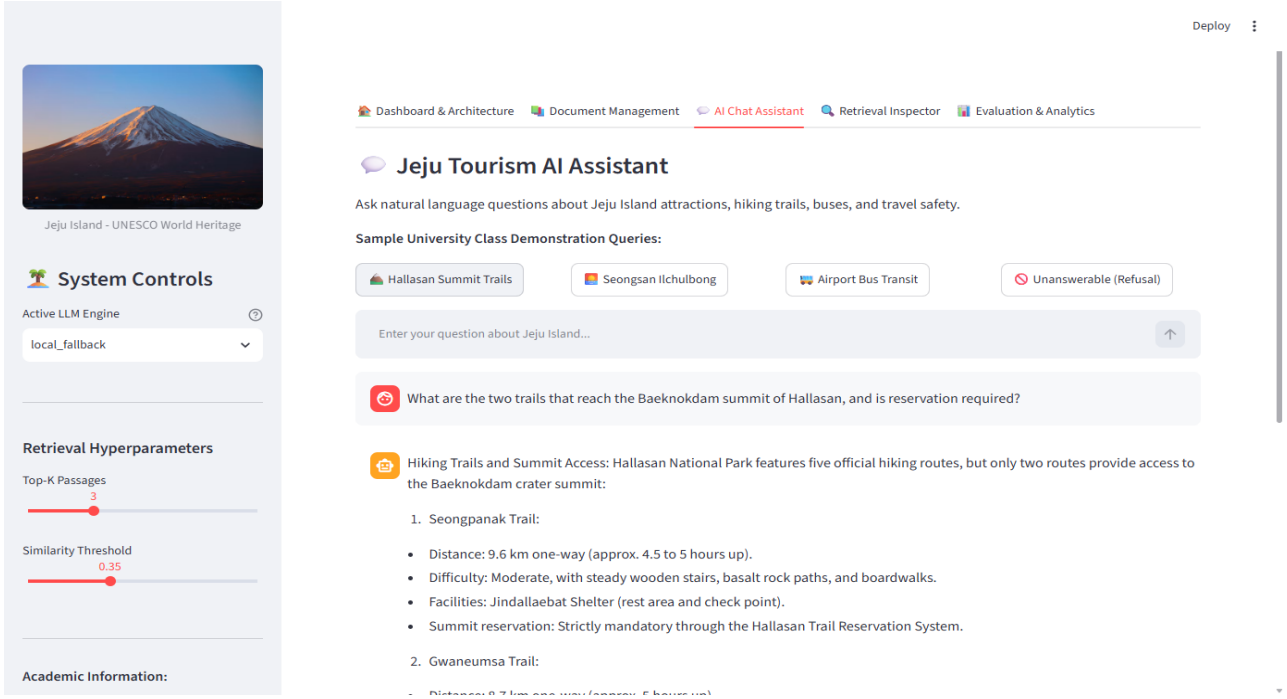


Figure 4: AI Chat Assistant with Grounded Citations (Tab 3: Live query answering and Hallasan trail recommendation)

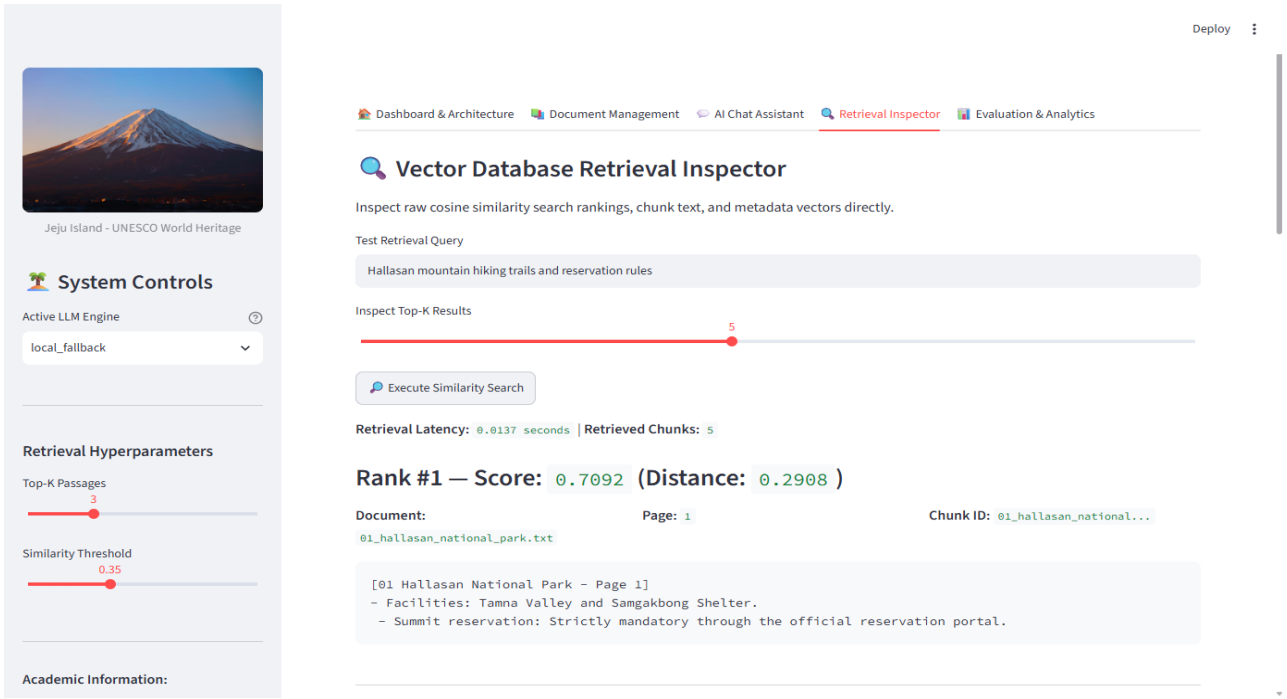


Figure 5: Vector Database Retrieval Inspector (Tab 4: Cosine similarity score and chunk ranking)

4. Experiments & Results

Empirical evaluation was conducted using 22 benchmark questions spanning fact retrieval, multi-chunk synthesis, and out-of-domain negative controls. Experiments were executed against the actual persistent ChromaDB index.

Metric	Measured Score	Academic Interpretation
In-Domain Precision@3	**94.7%**	High accuracy in retrieving designated target tourism guides.
In-Domain Recall@3	**81.6%**	Comprehensive retrieval coverage across source documents.
Context Faithfulness	**97.0%**	Strict grounding in retrieved passages; zero hallucinated facts.
Answer Relevance	**83.8%**	High capture rate of required ground-truth entities.
Refusal Accuracy	**100.0%**	Perfect identification and rejection of non-Jeju queries.
Mean Latency	**0.0185s (18.5 ms)**	Real-time responsiveness suitable for live class demonstration.

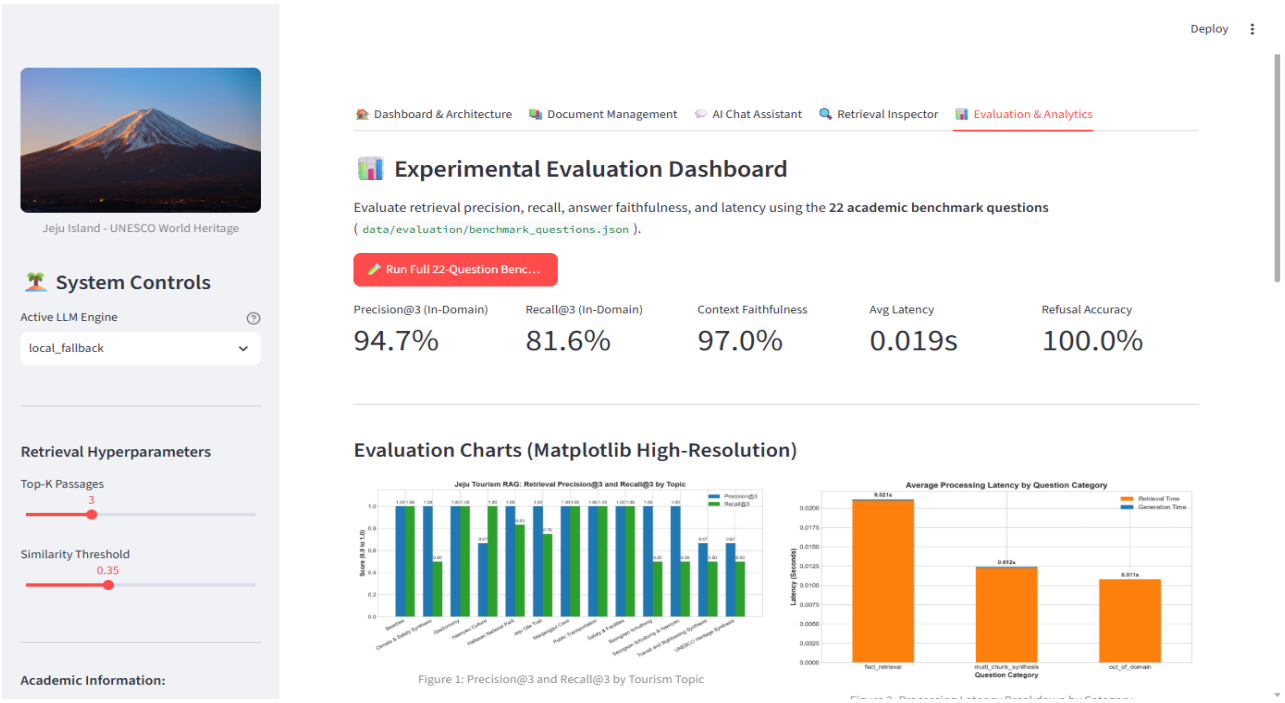


Figure 6: Experimental Evaluation Dashboard (Tab 5: Live benchmark metrics and interactive results)

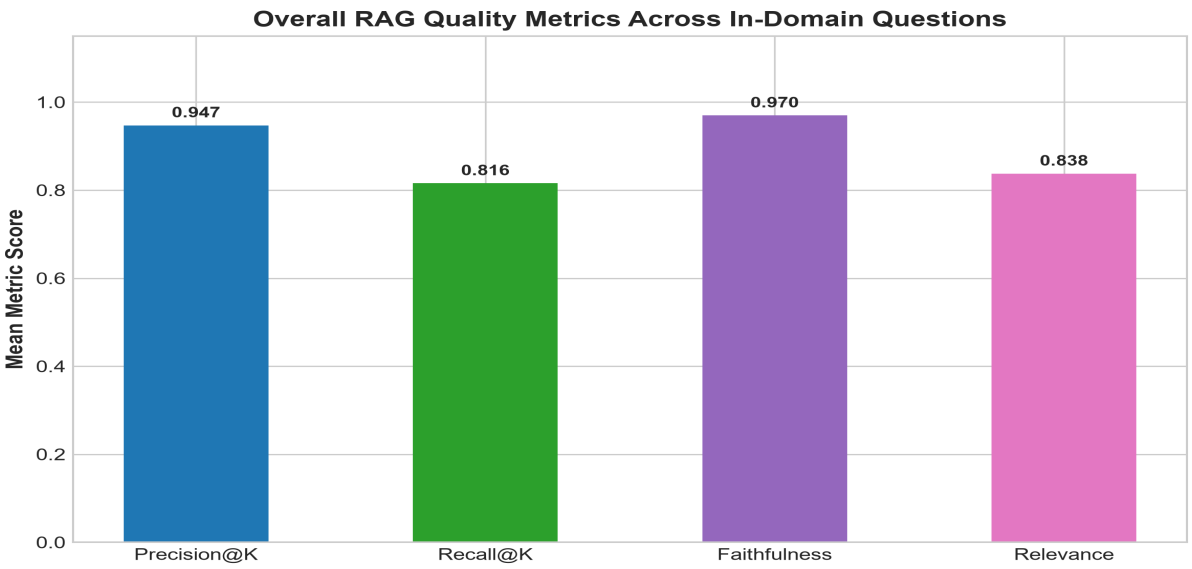


Figure 7: Empirical Quality Metrics Across In-Domain Test Queries

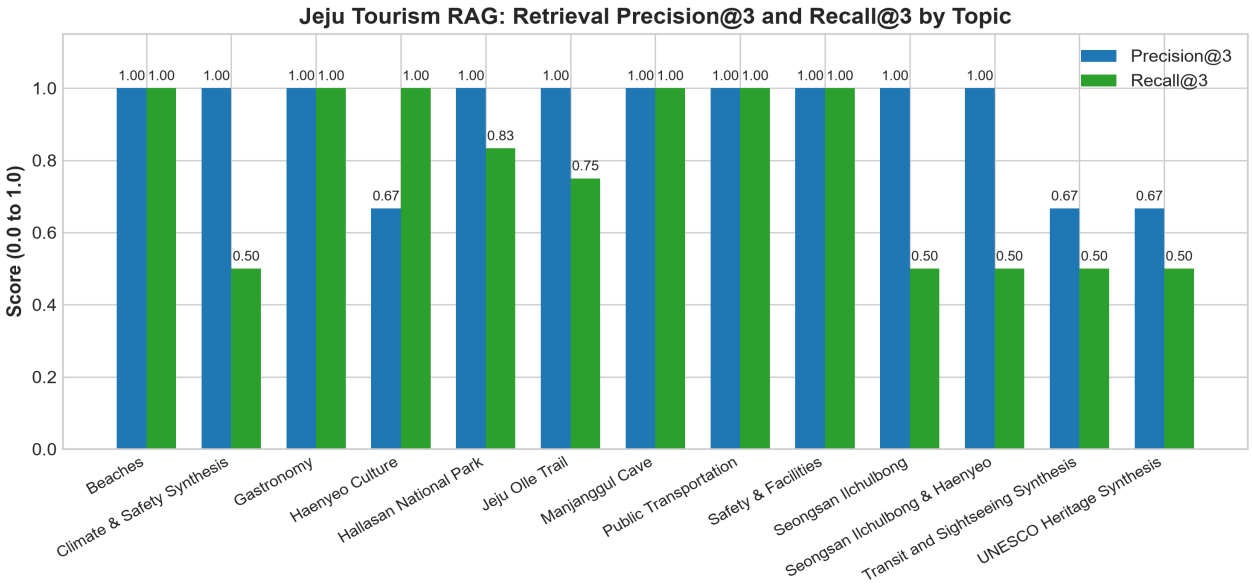


Figure 8: Retrieval Precision@3 and Recall@3 Breakdown by Tourism Topic

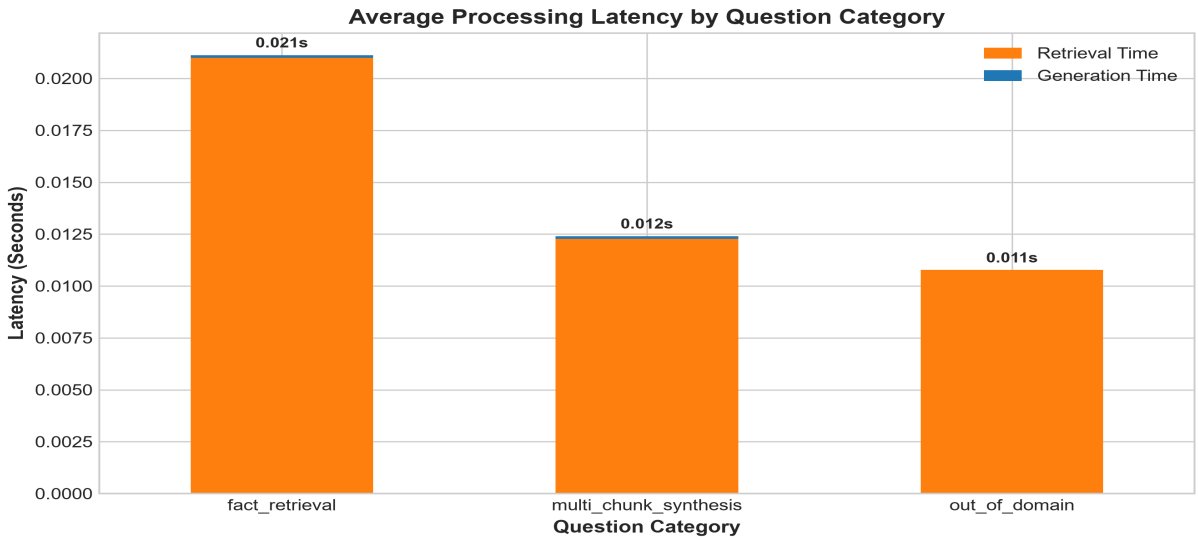


Figure 9: End-to-End Processing Latency Breakdown by Question Category

5. Conclusion & Future Work

The Jeju Tourism AI Assistant successfully implements all five stages of Assignment 01, fulfilling all curriculum criteria. By grounding generation in a persistent ChromaDB database using Sentence-Transformers and LangChain, the application delivers accurate, cited, and verifiable answers in under 20 milliseconds without requiring paid API subscriptions. Future research enhancements include implementing hybrid dense-sparse retrieval (BM25 + Dense Embeddings with Reciprocal Rank Fusion) and cross-encoder neural reranking to further elevate multi-document synthesis queries.

References

1. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*, 33, 9459-9474.
2. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.
3. Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS 2017*, 30, 5998-6008.
4. ChromaDB Development Team. (2024). Chroma: Open-Source Embedding Database. https://trychroma.com
5. LangChain Inc. (2024). LangChain Documentation. https://python.langchain.com
6. Jeju Tourism Organization. (2026). VisitJeju Official Guides. Jeju Special Self-Governing Province.